# Ejemplo de aplicación Multiplicación de matrices

Universidad Nacional de La Plata



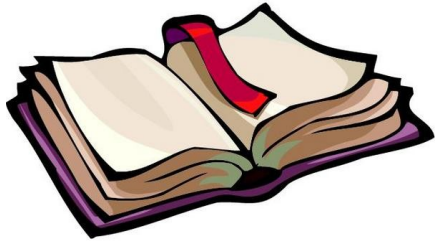
Facultad de Informática

Prof. Dr. Adrian Pousa

# Agenda



I. Introducción a la multiplicación de matrices
II. Organización de matrices en memoria
III. Solución en GPU



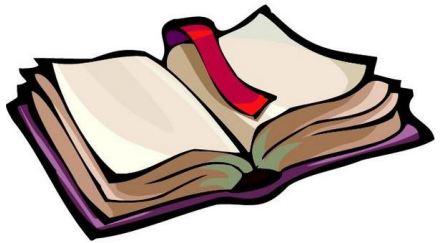
# Agenda



I. Introducción a la multiplicación de matrices

II. Organización de matrices en memoria

III. Solución en GPU



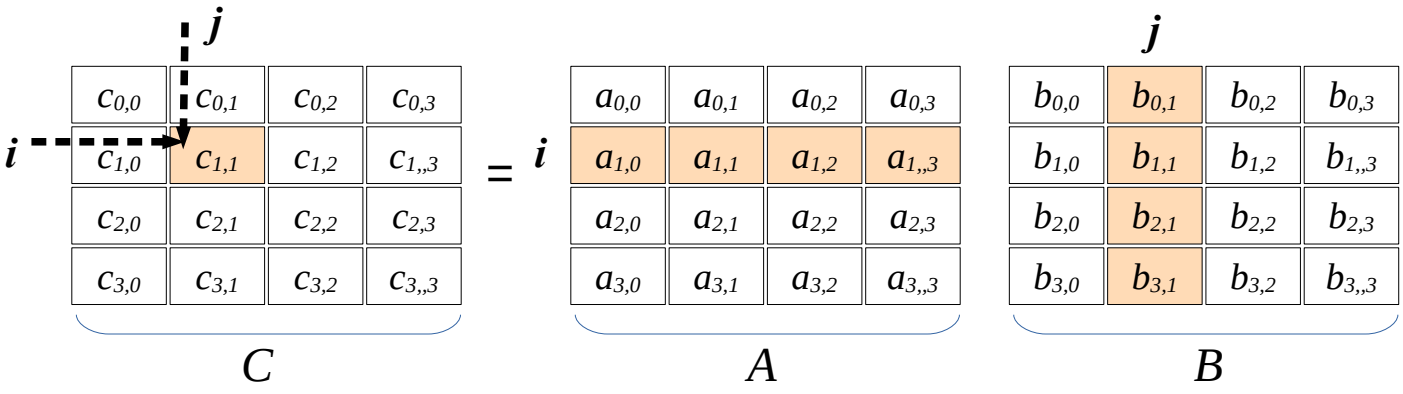
# Introducción a la Multiplicación de matrices



- Dadas dos matrices $A$ y $B$ de $NxN$ elementos. La multiplicación $AB$ retorna una matriz resultado $C$ donde cada elemento se calcula como:

$$c_{i,j} = \sum_{k=0}^{N-1} a_{i,k} \cdot b_{k,j}$$

- Para calcular la posición $(i,j)$ de la matriz resultante $C$ es necesario procesar la fila $i$ de $A$ y la columna $j$ de $B$.



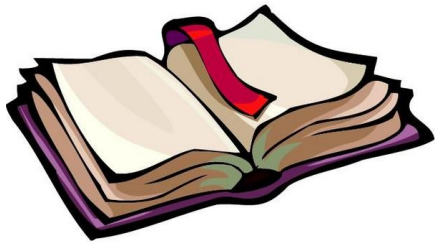
# Agenda



I. Introducción a la multiplicación de matrices

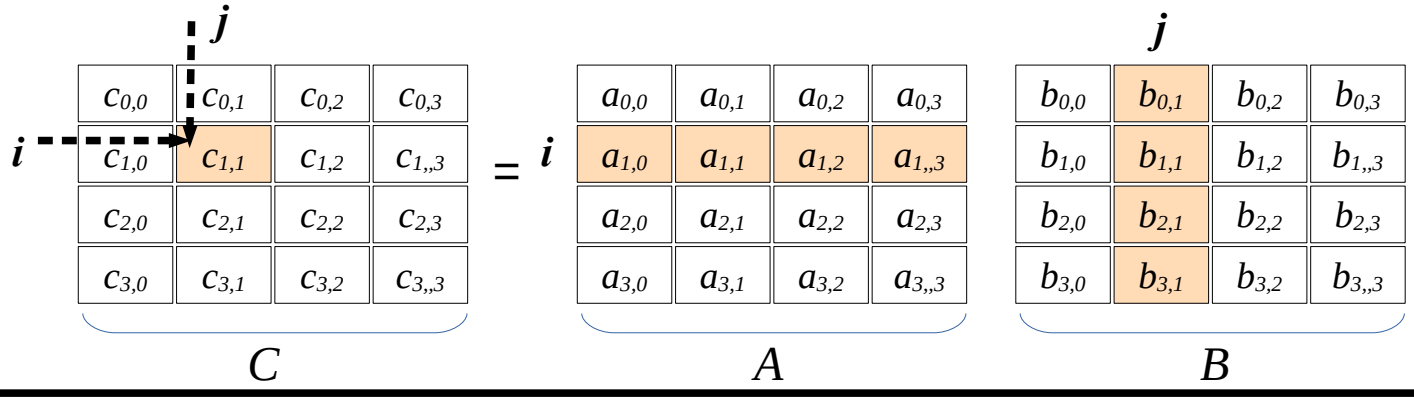
II. Organización de matrices en memoria

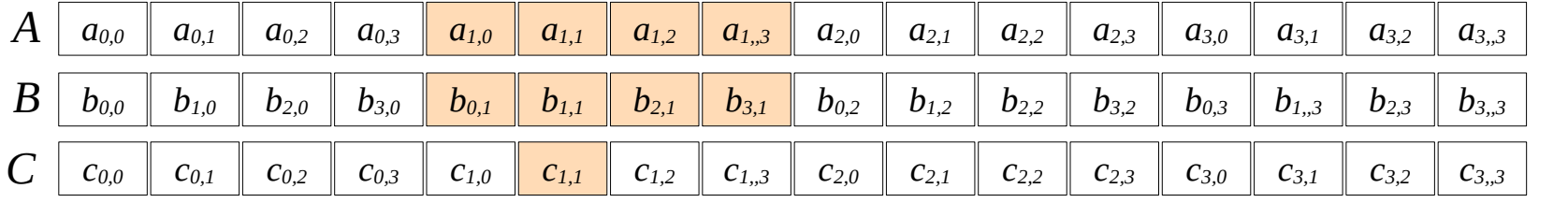
III. Solución en GPU



# Multiplicación de matrices
# Organización de matrices en memoria - Localidad

- Para mejorar la localidad de caché, las matrices se trabajan como arreglos ordenados en memoria de acuerdo al patrón en el cual se acceden:
  - Las matrices $A$ y $C$ se almacenan en memoria por filas, y $B$ por columnas.




| | |
|---|---|
| Acceso a la posición *(i,j)* de una matriz *M* ordenada por filas: | `M[i*N+j]` |
| Acceso a la posición *(i,j)* de una matriz *M* ordenada por columnas: | `M[i+j*N]` |

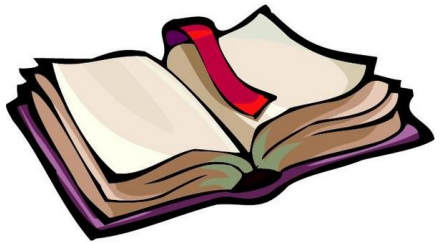
# Agenda



I. Introducción a la multiplicación de matrices

II. Organización de matrices en memoria

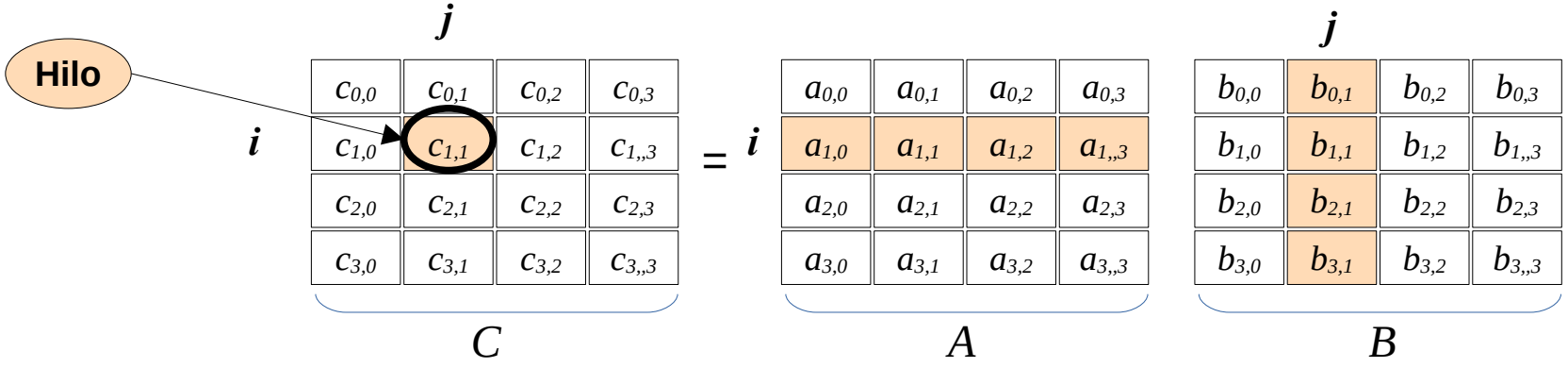
III. Solución en GPU



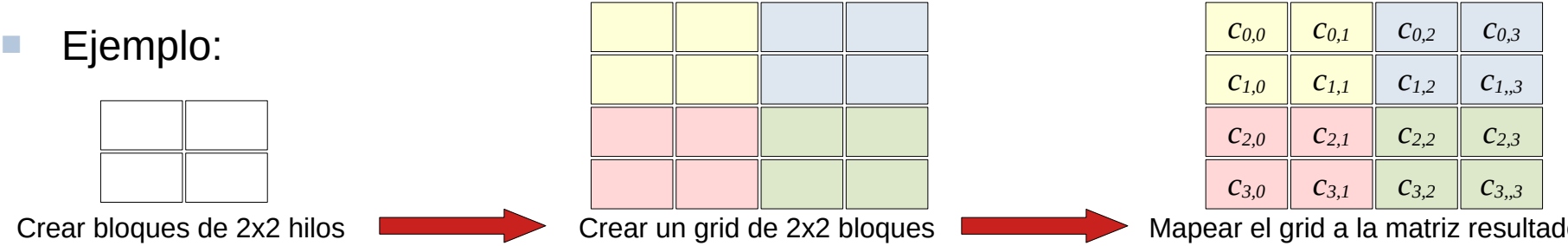
# Multiplicación de matrices
# Solución en GPU



- La solución en GPU asigna a cada hilo el cálculo de una celda de la matriz resultado.
- Cada hilo CUDA necesita acceder a la fila $i$ de $A$ y la columna $j$ de $B$.



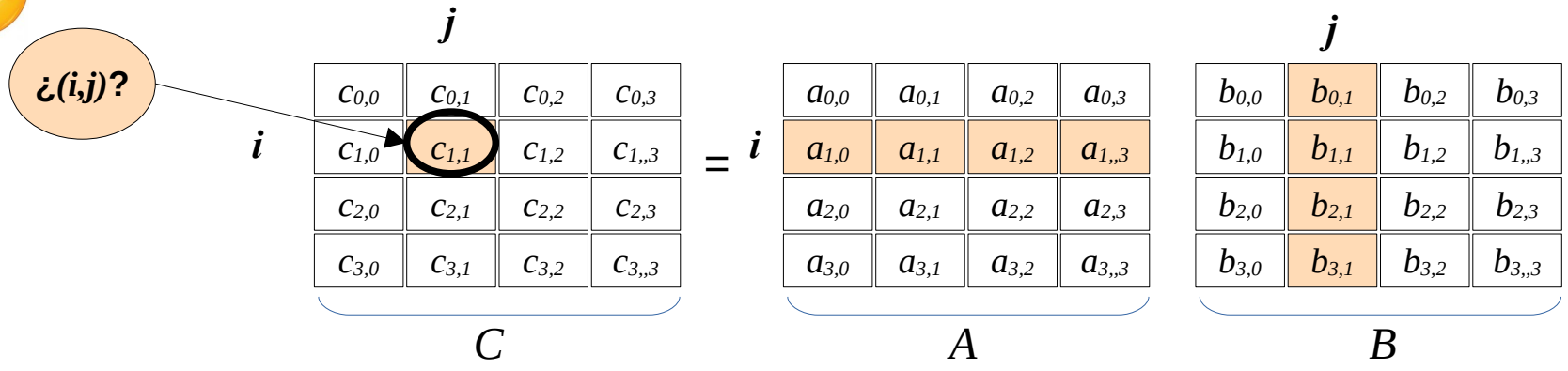

- CUDA permite crear Grids y Bloques bi-dimensionales de hilos que pueden mapearse a la estructura bi-dimensional de las matrices:
  - Ejemplo:



# Multiplicación de matrices
# Solución en GPU

## ¿Cómo conoce cada hilo CUDA la posición que debe calcular?



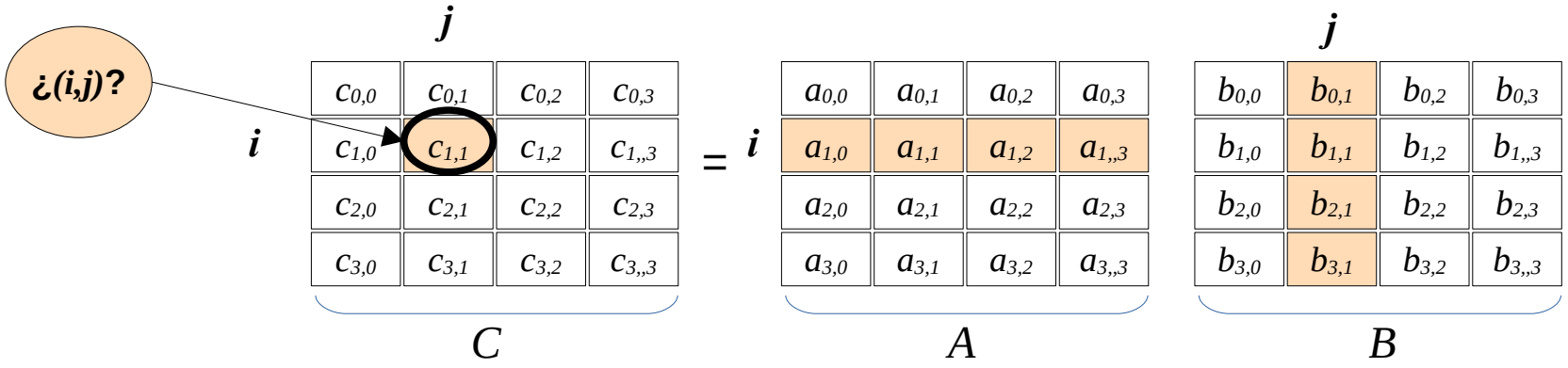


# Multiplicación de matrices
# Solución en GPU

**¿Cómo conoce cada hilo CUDA la posición que debe calcular?**



**Utilizando las variables built-in!!!**




- **En el Kernel**: código que calcula la posición *(i,j)*:

```
__global__ mm(int *C, int *A, int *B,int N){
int i = blockIdx.y*blockDim.y + threadIdx.y;
int j = blockIdx.x*blockDim.x + threadIdx.x;
int k;
    for( k=0 ; k<N ; k++ )
     C[i*N+j] +=  A[i*N+k] * B[k+j*N];
}
```

# Multiplicación de matrices
# Solución en GPU

**Si la dimensión del grid no es proporcional al tamaño de la matriz (más hilos que posiciones a calcular de la matriz $C$) existen hilos que NO deberían trabajar.**
**Se agrega el siguiente if al código:**



```
__global__ mm(int *C, int *A, int *B,int N){
int i = blockIdx.y*blockDim.y + threadIdx.y;
int j = blockIdx.x*blockDim.x + threadIdx.x;
int k;
     if ( i<N && j<N ){
       for( k=0 ; k<N ; k++ )
          C[i*N+j] +=  A[i*N+k] * B[k+j*N];
     }
}
```



